

レポート課題 4

05-191009 奥村真司

問 1

動作例

```
# let table = [("x", 6); ("y", 0); ("z", 2)];;
val table : (string * int) list = [("x", 6); ("y", 0); ("z", 2)]
# lookupDiv "x" "y" table;;
- : int myErr = Err "Div by Zero"
# lookupDiv "x" "z" table;;
- : int myErr = Ok 3
# lookupDiv "x" "b" table;;
- : int myErr = Err "Not found"
# lookupDiv "a" "z" table;;
- : int myErr = Err "Not found"
```

考察

授業スライドの例 1 を参考に実装した。bind は 0 除算や lookup が失敗したなどの例外が発生したらそれを伝播し、それ以外では通常のように計算する。

問 2

動作例

```
- : (int * int * int * int * int) list =
[(0, 0, 0, 0, 0); (1, 2, 0, 4, 4); (2, 4, 0, 8, 8); (2, 5, 0, 1, 0);
 (3, 7, 0, 5, 4); (4, 9, 0, 9, 8); (5, 0, 1, 0, 0); (6, 2, 1, 4, 4);
 (7, 4, 1, 8, 8); (7, 5, 1, 1, 0); (8, 7, 1, 5, 4); (9, 9, 1, 9, 8)]
- : (int * int * int * int * int * int * int * int) list =
[(9, 5, 6, 7, 1, 0, 8, 2)]
```

考察

1 つ目の覆面算ではなんの枝刈りもせず全ての可能性を探索して、最後に条件を満たすものを取り出している。2 つ目の覆面算では 0 が禁止されているものについては 0 を、それとそこまでで他の文字に割り振られた数字を候補から外すという制約的にごく自然な枝刈りを施しただけで実行時間は 2 秒ほどになった。

問 3

動作例

```
type 'a m = 'a * string
val ( >=> ) : 'a m -> ('a -> 'b m) -> 'b * string = <fun>
val return : 'a -> 'a * string = <fun>
val writer : string -> unit * string = <fun>
val msg : int -> string = <fun>
val fib : int -> int m = <fun>
Fib(4)
Fib(2)
Fib(0)
Fib(1)
Fib(3)
Fib(1)
Fib(2)
Fib(0)
Fib(1)
- : unit = ()
```

考察

モナドは計算結果と出力バッファの文字列の組とした。bind はバッファに文字を追加し、return は空文字列のバッファを付加する。動作例ではフィボナッチ数をもとめる再帰関数 Fib の再帰呼び出しのコール順を表示している。深さ優先で呼ばれていることと、0 や 1 などが何度も呼ばれており無駄があることも見て取れる。同じ引数で何度も呼ぶ無駄は、問 4 のメモ化によって解決される。

問 4

動作例

```
# fib 80 [];;
- : int * (int * int) list =
(23416728348467685,
 [(79, 14472334024676221); (78, 8944394323791464); (77, 5527939700884757);
  (76, 3416454622906707); (75, 2111485077978050); (74, 1304969544928657);
  (73, 806515533049393); (72, 498454011879264); (71, 308061521170129);
  (70, 190392490709135); (69, 117669030460994); (68, 72723460248141);
  (67, 44945570212853); (66, 27777890035288); (65, 17167680177565);
  (64, 10610209857723); (63, 6557470319842); (62, 4052739537881);
  (61, 2504730781961); (60, 1548008755920); (59, 956722026041);
  (58, 591286729879); (57, 365435296162); (56, 225851433717);
  (55, 139583862445); (54, 86267571272); (53, 53316291173);
  (52, 32951280099); (51, 20365011074); (50, 12586269025); (49, 7778742049);
  (48, 4807526976); (47, 2971215073); (46, 1836311903); (45, 1134903170);
  (44, 701408733); (43, 433494437); (42, 267914296); (41, 165580141);
  (40, 102334155); (39, 63245986); (38, 39088169); (37, 24157817);
  (36, 14930352); (35, 9227465); (34, 5702887); (33, 3524578); (32, 2178309);
  (31, 1346269); (30, 832040); (29, 514229); (28, 317811); (27, 196418);
```

```
(26, 121393); (25, 75025); (24, 46368); (23, 28657); (22, 17711);  
(21, 10946); (20, 6765); (19, 4181); (18, 2584); (17, 1597); (16, 987);  
(15, 610); (14, 377); (13, 233); (12, 144); (11, 89); (10, 55); (9, 34);  
(8, 21); (7, 13); (6, 8); (5, 5); (4, 3); (3, 2); (2, 1); (1, 1); (0, 0)]]
```

考察

メモ化テーブルを状態として持つので、授業スライドの例 3 を参考に実装した。まずモナドの型は前のメモ化テーブルの状態から、計算結果と新しいメモ化テーブルの状態の組への関数、すなわち $((\text{int} * 'a) \text{ list}) \rightarrow ('a * ((\text{int} * 'a) \text{ list}))$ とした。bind, return は授業スライドの例 3 とほぼ同様である。memo は、メモ化して計算したい関数 f とその引数 n を引数にとって、実際にメモ化しながら計算する関数である。戻り値の型はモナドなので、前のメモ化テーブル (problem4.ml 内では 13 行目の x) を受け取って、その x の中に引数が n の場合の結果が格納されていればその値とテーブルの組を返し、そうでない場合は実際に計算して、その計算結果とそれを書き込んだ新しいテーブルの組を返す。runMemo は初期テーブルとして空リストを実際に与えて、帰ってくる計算結果とテーブルの組から計算結果を取り出す関数である。memo と runMemo の実装は先述の説明をコードに自然に落とし込んだものである。動作例では fib 80 の結果とメモ化テーブルの様子を示した。

発展 1

動作例

```
# runGenerate 10 3;;  
- : int list = [79; 101; 14; 93; 24; 21; 114; 87; 91; 86]  
# runGenerate 15 11;;  
- : int list = [67; 116; 25; 109; 4; 46; 53; 74; 18; 88; 60; 95; 81; 39; 32]  
# runGenerate 15 7;;  
- : int list = [21; 114; 87; 91; 86; 3; 77; 44; 115; 56; 100; 45; 84; 65; 59]
```

考察

0~118 の疑似乱数を生成するモナドを実装した。疑似乱数は seed 値から適当な式で算出される数列であり、実装では最後に生成された値を次の乱数生成の seed 値として用いている。実装方針は状態をもつモナドと同様で、モナドの型は初期 seed 値から計算結果 (生成された乱数のリスト) と次に乱数を生成する際に用いられる seed 値の組への関数である。runGenerate は 2 つ引数を取り、1 つ目は生成したい乱数の個数、2 つ目が初期 seed 値である。

発展 2

動作例

```
# #use "advanced2.ml";;  
type 'a m = 'a list  
val single : 'a list -> bool = <fun>
```

```
val all : ('a -> bool) -> 'a list -> bool = <fun>
val join : 'a list list -> 'a list = <fun>
val ( >=> ) : 'a m -> ('a -> 'b m) -> 'b list = <fun>
val return : 'a -> 'a list = <fun>
```

考察

<https://qiita.com/1to100pen/items/1558a4d6600c47f7f859> このページを参考にして実装した。実装はしたがどのような計算を表現しているかはあまり理解できなかった。